

Cassandra: catching AI agents when they make things up

A plain-English deep-dive into what we built, how it actually works, whether it works, who would pay for it — and, honestly, where it falls short.

Team Kshitij & Sirjan Origin Google Cloud Rapid Agent Hackathon (Arize track, 1st)

Status working prototype, pre-users Read time ~15 min

CONTENTS

- | | |
|-------------------------------|---------------------------|
| 01 The short version | 07 When it can't be sure |
| 02 The words, defined | 08 The full pipeline |
| 03 The problem | 09 Does it actually work? |
| 04 What we're really building | 10 Honest limitations |
| 05 How it works | 11 Market & buyers |
| 06 The checker (the moat) | 12 Business model |
| | 13 The verdict |

01 The short version

AI assistants that answer customer questions have one expensive habit: when they don't know something, they don't say so — they *confidently make it up*. Cassandra is a system that watches those assistants and catches the made-up answers, **with proof**, before a customer sees them.

The important honesty up front: our value is **not** "we rewrite your prompts." Rewriting a prompt is easy and nobody pays for it. The value is **detection, proof, and prevention** — catching an unsupported answer using a reproducible rule (not another AI's opinion), turning it into a permanent test, and blocking it from coming back.

~\$2.7B

market we sit in (LLM observability + eval), ~36%/yr growth

0% → 100%

real fix, measured: bad-answer rate on a synthesized test set

1 of ~14k

hackathon result the prototype came from

THE ONE SENTENCE

Deterministic *faithfulness* checking for tool-using agents: for every claim the bot makes, is there a real tool result that backs it up? — a rule with receipts, where competitors use an AI's guess.

02 The words, defined

Six terms carry this whole report. In plain English:

Tool	A little function the AI can use to look something up — <code>get_fee()</code> , <code>get_order()</code> . The AI itself knows nothing about your business; tools fetch the real data.
Tool ledger	The record of which tools the bot called and what they returned — the <i>receipts</i> . e.g. <code>get_fee</code> → <code>{found:false}</code> .
Grounded	The answer stands on evidence — it said "\$15" <i>and</i> a tool returned \$15.
Ungrounded	The answer is floating in the air — a claim with no receipt behind it. The bot made it up.
Hallucination	The headline failure: an ungrounded, confidently-stated claim.
Faithfulness	The real, named metric: does the answer stick to its sources? We check faithfulness — <i>not</i> truth (more on that below).

03 The problem

An AI agent doesn't crash when it fails. It doesn't throw an error. It just... invents something — in a warm, friendly tone — and ships it to a customer.

Take a support bot. A customer asks "*what's the refund policy for Germany?*" The bot checks its records, finds nothing, and — rather than admit it — replies "*Germany has a 14-day return policy!*" That number is completely invented. Nothing turns red. The only trace of the mistake is buried in the logs: the lookup came back empty, yet the bot answered as if it knew.

Today this is caught by **humans** — reading conversations one by one, writing test cases by hand, editing prompts on gut feeling. It doesn't scale, and most failures are never caught. In regulated industries (finance, insurance, healthcare) one confidently-wrong answer is a compliance event, not just a bad day.

WHY NOW

Research is blunt about it: even retrieval-augmented (RAG) systems still hallucinate 40–80% in the hard cases, and *better retrieval doesn't fix faithfulness*. As more companies ship agents, the "did it actually stick to its data?" problem grows, not shrinks.

04 What we're really building

It took a few brutal pitch sessions to find the honest framing. Three realizations reshaped the product:

1. We don't know the truth — and we don't claim to.

Cassandra has no idea what Germany's real refund policy is. What it checks is narrower and *solvable*: is every claim backed by the agent's **own retrieved evidence** (its tool results)? The source of truth is the tool, never us. If the tool itself returns wrong data, we can't catch that — and we say so.

2. "Just tell the bot to say 'I don't know'" isn't a fix.

The obvious objection: why not prompt the bot to decline when unsure? Because a model told to abstain *still* fabricates — it doesn't *know* it's unsure; it's confidently wrong. The instruction is the easy part. **Proving it's obeyed, on live traffic, is the hard, valuable part.** Cassandra is the test and the audit, not the code.

3. Detection beats auto-fix.

People pay to avoid pain — a wrong answer that costs money, an audit they can't pass, a silent regression. They don't pay for "a nicer prompt." So we lead with the **verdict + evidence + a number** (the bad-answer rate), and the auto-fix is a footnote.

REFRAME

From "an AI that fixes your agents" → "a **grounding verifier that catches hallucinations, with proof, for tool-using agents.**" Smaller, less flashy, harder to say — but it survives every hard question.

05 How it works

The core design rule: **Cassandra never touches the agent it watches.** It learns everything from the logs (traces) the agent already emits — like a security guard watching the camera feeds instead of following you around the store.

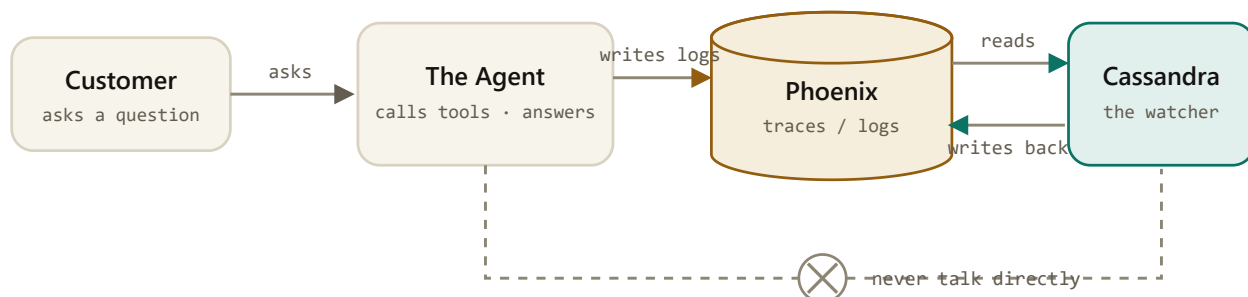


Figure 1 — Two separate agents that communicate only through the logs. Point Cassandra at any agent that emits traces; the demo bot ("ShopBot") is just one example.

Mechanically, Cassandra **polls** the log store for new entries, through a single "gateway" file (one door everything passes through). It talks to the store over **MCP** — a standard plug (like USB) for tools — using a middleman program that makes the actual network calls, so our code stays insulated from the store's details. Each log entry (a *span*) carries the question, the answer, and the tool ledger. That ledger is the raw material for everything next.

06 The checker — the moat

Here is the one genuinely defensible idea. Instead of asking *another AI* "does this look like a lie?", we run a plain rule: **for every specific claim in the answer, is there a successful tool result that backs it?** — and we cite the exact call as proof.

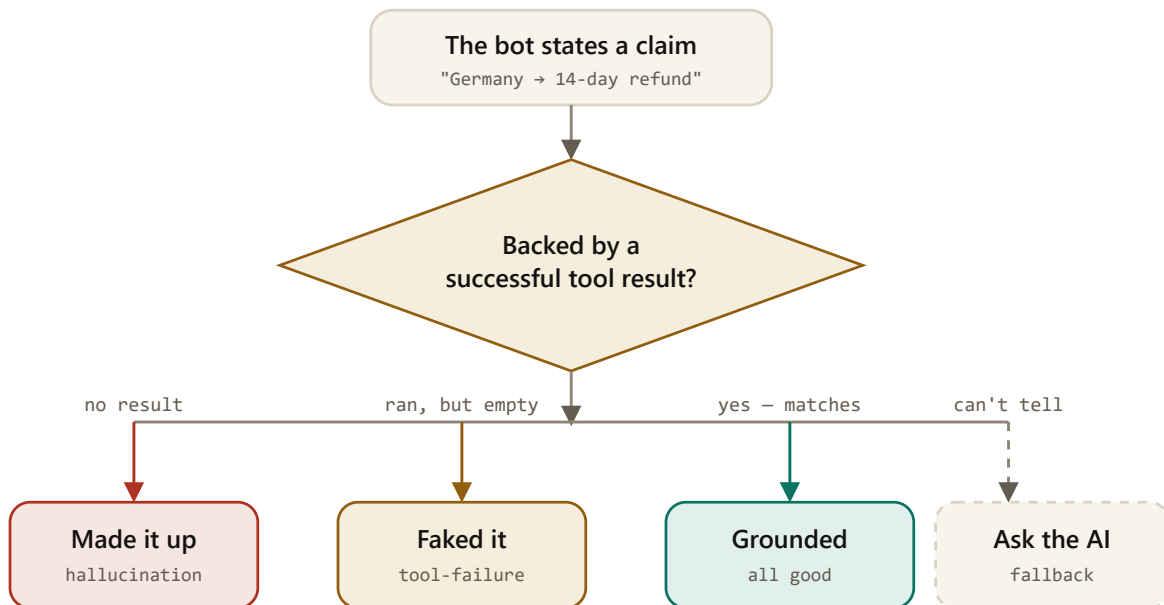


Figure 2 — The deterministic verdict. Reproducible, free, and it can point at the exact tool call as evidence. Only the fuzzy cases (dashed) go to an AI.

It's not hardcoded to one bot. The rule engine is generic; each agent brings a tiny config (a "grounding spec") — ~15 lines saying which tool backs which claim, plus regexes to spot claims (a dollar amount, a date, a carrier). A bank spec and an airline spec differ only in those values. Swap the spec, same engine.

WHY THIS IS THE MOAT

Almost every competitor checks faithfulness with *another AI* — which reopens the "how do you know the judge is right?" problem. When the evidence is a structured tool result, we answer it with a **rule that shows its receipt**. That is hard to copy and easy to trust.

07 When it can't be sure

The rule doesn't always apply — and the system knows it. Two honest fallbacks:

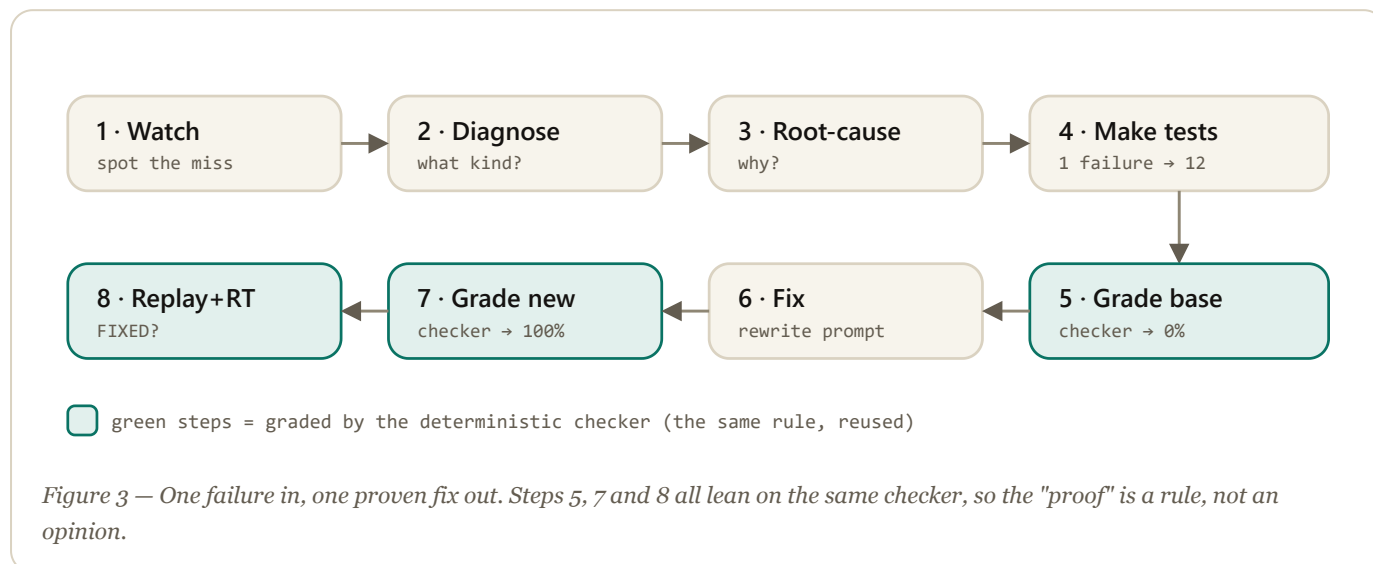
- **No ledger.** Some agents don't report their tool calls. No receipts → the rule can't run → the *whole answer* goes to an AI judge, which grades it against the expected behavior instead of against evidence.
- **Abstain.** Even with a ledger, if a claim matches no rule, the checker returns "*abstain*" — "I honestly can't decide this one" — and hands it to the AI. It never guesses.

The AI judge is just a prompt: a rubric plus the case, and the model returns a pass/fail opinion. It works, but it's fuzzy, non-deterministic, and can't cite proof — which is exactly why it's the *backup*. We also have an optional **jury** (several AI judges vote, and their agreement is reported), off by default. And there's a self-check

that grades Cassandra's own diagnoses against a hand-labeled answer key — a nice introspection feature, but small and peripheral, not part of the core flow.

08 The full pipeline

When an ungrounded answer is caught, an 8-step loop turns that one failure into a proven fix. The key insight: **the same deterministic checker is reused as the grader** across the scoring steps.



THE FIX NEVER INVENTS TRUTH

A crucial point people miss: Cassandra never fills in "the right value." If the tool returned *nothing*, there is no right value — so the fix makes the bot **decline honestly** ("that's unavailable, contact support"). If the tool returned a value the bot misreported, the fix makes it **report the tool's value faithfully**. The only truth used is the tool's own output.

09 Does it actually work?

We ran it end-to-end on the demo bot. Here is the real scorecard — and, more tellingly, a run where the fix was *bad* and the system caught it.

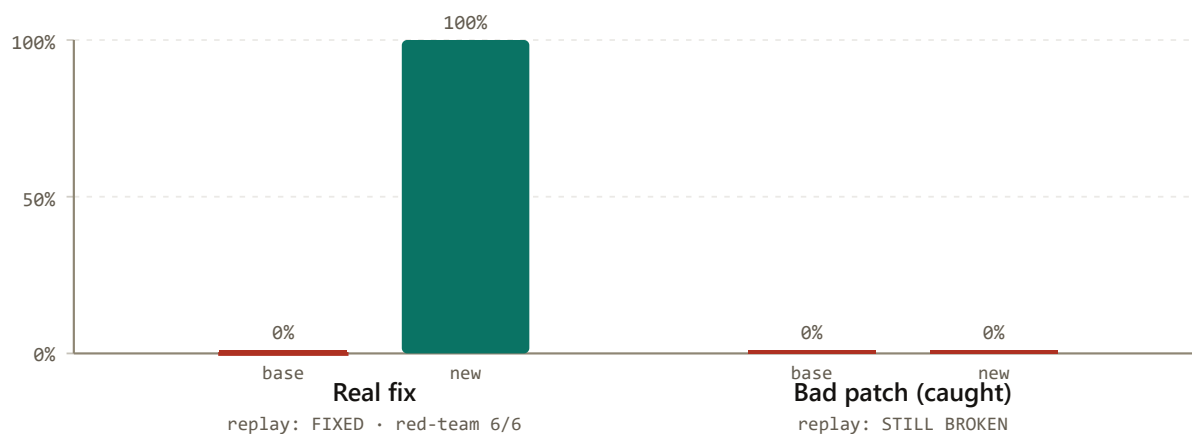


Figure 4 — Left: a genuine fix drives the bad-answer rate 0%→100% pass. Right: an earlier run produced a bad "fix" (it swapped one invented number for another) — and the scorecard refused to bless it: 0%→0%, replay STILL BROKEN.

METRIC	BEFORE	AFTER	WHAT IT PROVES
Pass rate (12-case test set)	0%	100%	the fix generalizes past the one case
Replay of the original failure	fabricated	declines honestly	the exact real failure is gone
Red-team survival	0 / 6	6 / 6	holds under adversarial probes
Latency	—	-22%	cheaper, not just safer

THE STRONGEST EVIDENCE

The system caught its *own* bad patch. A fake metric would have said "fixed!"; ours said 0%→0%, STILL BROKEN. That's the difference a deterministic, receipt-backed grader makes — it can't be fooled by an authoritative-sounding wrong answer.

10 Honest limitations

An honest review is mostly this section. Where it's weak:

1 · IT ONLY SHINES ON STRUCTURED TOOLS

The deterministic check is precise when the evidence is a clean field (`{found:false}`, `{gate:"A4"}`) — fees, statuses, dates. For **free-text document RAG** ("is this sentence supported by this paragraph?"), matching is fuzzy and falls back to an AI. Our defensible territory is *tool-calling, fact-serving agents* — narrower than "all AI agents."

2 · SOMEONE HAS TO WRITE THE SPEC

Each agent needs a hand-written grounding spec (the regexes + rules). That's real onboarding friction. "Auto-generate the spec from traces" is the single highest-leverage thing we could build — and it doesn't exist yet.

3 • WE CHECK FAITHFULNESS, NOT TRUTH

If the tool itself returns wrong data, the answer is "faithful" and we pass it. We catch *fabrication beyond the sources*, not *wrong sources*. Be upfront about this in every pitch.

4 • THE CLASSIFIER IS STILL AN AI

Deterministic grading decides pass/fail, but labelling the *type* of failure still uses an AI judge. Better than before (it now sees the receipts), but not fully deterministic.

5 • "IT WORKS" IS MEASURED ON OUR OWN TESTS

The pass-rate is scored against a test set Cassandra synthesized. Two things break the circularity — the grading is deterministic, and the replay re-tests the *real* original failure — but "are the synthesized tests representative?" is a fair open question (an RL adversary that hunts new failures would help).

6 • AUTONOMY IS NOT THE PRODUCT

No company will let an AI edit production prompts unsupervised. The auto-fix must stay human-approved. That's fine — but it means "fully autonomous" is a demo word, not a sellable one.

7 • PROTOTYPE, NOT A COMPANY

It's a strong hackathon build with zero users. Real competitors (below) are funded and shipping. The moat is thin unless the deterministic edge + an owned "regression-dataset flywheel" compound over time.

11 Market & buyers

The market — LLM/agent observability & evaluation — is real and fast-growing (~\$2.7B in 2026, ~36%/yr, and Gartner expects it in half of GenAI deployments by 2028). But the "watch & evaluate" seat is crowded; the "catch & prove faithfulness" angle is where we fit.

Who actually buys

Not "companies with AI agents." Specifically: a **platform/ML lead or head-of-support at a company where a wrong answer costs money or triggers compliance** — fintech, insurance, healthcare, regulated support — already running a tool-using agent in production. That person has both budget and fear.

LAYER	MEANING	SIZE
TAM	All LLM observability + evaluation	~\$2.7B (→ ~\$9B by 2030)
SAM	Tool-using, high-stakes agents needing faithfulness + gating	~\$0.8–1.2B
SOM	Realistic 1–3 yr reach via the free-gate wedge	~\$3–10M ARR

The competition is real (and named)

PLAYER	WHAT THEY DO	OUR EDGE
RAGAS / TruLens	open-source faithfulness scoring (AI-graded)	deterministic for structured tools
Arize / Galileo	observability + hallucination evals, guardrails	act on it: detect→prove→gate loop
Patronus AI	hallucination detection for regulated finance	rule-with-receipt vs. model-based; open core

SOBER NOTE

Patronus already sells "hallucination detection mapped to regulatory evidence" to finance — our exact vertical. This is not a blue ocean. We win only on the *deterministic-for-tools* wedge + open-source adoption + the owned failure-case flywheel, or we don't win.

12 Business model

Mirror the category — it's proven:

- **Open-source core** (the checker + a CI "prompt gate") → adoption, trust, inbound. This is the low-friction wedge: *"regression-test your agent's answers, fail the build if the bad-answer rate rises."*
- **Paid hosted / enterprise** → a monitoring dashboard, the owned dataset store, team features, and the highest-value tier: **audit & compliance evidence** ("prove every claim was grounded").
- **Meter on the valuable verb** — agents monitored / regression suites maintained — not raw log volume, so we're not just reselling storage.

THE ONLY METRIC THAT MATTERS RIGHT NOW

Not pricing. It's: can we get **one** team running a tool-calling agent to say *"Cassandra caught an ungrounded answer I actually cared about, and I'd pay to not miss the next one"*? Business model follows product-market fit — not before it.

13 The verdict

Honestly: the *problem* is real, big, and unsolved; the *market* is real and growing; the *method* (deterministic faithfulness) is genuinely differentiated — but only for tool-using agents, and only if we do the unglamorous work of removing onboarding friction and landing a first believer.

What must be true for this to become a company, in order:

1. **Narrow the story** to "deterministic faithfulness for tool-calling agents." It survives the hard questions; the broad version doesn't.
2. **Kill the onboarding friction** — auto-generate the grounding spec from traces.
3. **Land one design partner** in fintech/insurance/health and get the sentence above.
4. **Compound the flywheel** — every caught failure becomes a permanent, owned regression test; after six months that corpus is the switching cost.
5. Only then: hosted plane, compliance tier, pricing.

ONE LINE

We don't detect truth — we detect when an agent claims something its own tools never confirmed, prove it with a receipt, and stop it coming back. That's small, honest, and defensible. Everything else is execution.

Cassandra · internal product research — an honest review. Figures are from real prototype runs; SAM/SOM are reasoned estimates, TAM is from cited market reports. Written to be understood, not to impress.